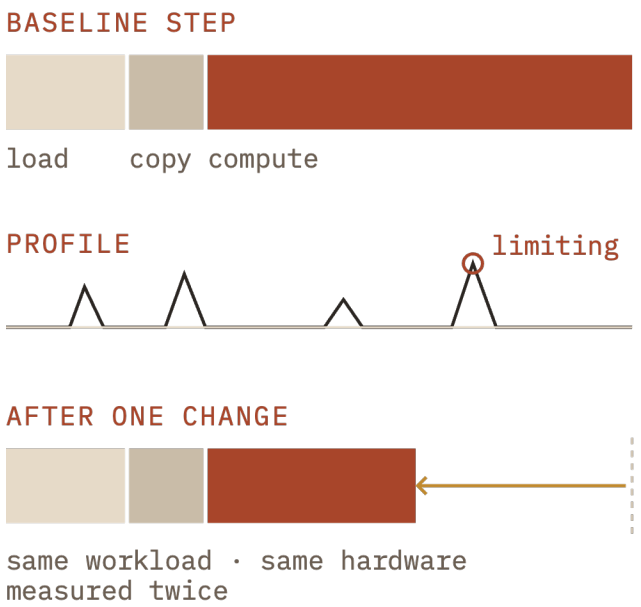
HIGH-PERFORMANCE MACHINE LEARNING · FALL 2026

Final Project

A measured performance study on modern AI systems

Assignment title: performance optimization of a machine learning model on accelerator-enabled infrastructure

Dr. Kaoutar El Maghraoui Columbia University



Outline

01 What the project requires — goals, the work sequence, teams

02 Model, data, hardware and scope — what you can actually run

03 Project ideas — plus your own

04 Deliverables, grading and dates

05 Prior projects — exemplars from last spring

PROPOSAL DUE 9 OCTOBER Teams of 4 to 5

Project goals

01 Apply high-performance ML techniques to a real workload

02 Profile before choosing an optimization

03 Work with real runtimes, tools and infrastructure

04 Carry out a systematic before and after performance study

A good project makes one specific performance claim and backs it with measurements someone else can repeat.

The work sequence

The performance-engineering loop from Lecture 1, applied over six weeks

01

Baseline

A working unoptimized run you can return to

02

Profile

Preprocessing, loading, transfer, training, inference

03

Optimize

One to three techniques you can defend against the profile

04

Re-measure

Same workload and configuration, logged

05

Validate

Show the output or task quality is unchanged

06

Explain

Say which resource was limiting and why the change helped

`Report with every number: hardware · dtype · batch/context · warm-up · repetitions · baseline`

Teams

Teams of 4 to 5. Individual contribution and collaboration quality are both evaluated.

Write the roles down	Roles, responsibilities, how you will communicate, and how you will resolve disagreement. State these in the proposal.
Pick complementary skills	Look for people whose strengths differ from yours and who are interested in the same model or system.
Meet on a schedule	Regular meetings to track progress and surface problems while there is still time to fix them.
Make contributions visible	Use the repository, the contribution statement and the project artifacts to make each member's work clear. Commit history is one source of evidence used to understand individual contribution.

What to build on

Three decisions to make in the proposal

MODEL / DATA

Choose something you can run repeatedly. Any data type: language, image, video, audio, gesture, physiological signals. Use an open dataset and a reproducible implementation.

INFRASTRUCTURE

Choose hardware you can access consistently enough to run matched benchmarks. Columbia research computing, cloud GPUs, TPUs, or edge devices such as Jetson.

SCOPE

One system. One or two questions. One to three techniques. Six weeks.

Bring the scope to office hours before you commit to it.

A SCOPE THAT FITS

One small language model, served locally. One question: where decode time goes at batch 1 compared with batch 16. Two techniques, both chosen from the profile.

A small model you can profile well usually makes a better project than a large one you can barely run.

GPU resources

This project can be completed on free resources

01

Columbia research computing

Shared Slurm-scheduled HPC clusters, including Insomnia. The right choice for anything running longer than a notebook session. Access instructions will be posted on CourseWorks.

02

Free notebook resources

Colab, Kaggle, Lightning AI Studios and Hugging Face Spaces. Past projects have been completed entirely on these. Sessions time out and get reclaimed, so avoid long fine-tunes.

03

Paid and cloud programs, optional

Per-hour GPU rental and cloud education credits. Entirely optional, and never expected. The class also has a $100 Google Cloud coupon per student. Details on the next slide.

· record the exact GPU · keep hardware fixed across before and after · free resources are acceptable

· nobody is expected to pay

Cluster access instructions will be posted on CourseWorks

Optional paid options

Nothing here is required. It is here in case you decide to rent

FOR THIS CLASS Every student receives a Google Cloud coupon worth $100. Redemption instructions go on CourseWorks. Stop your instances when you are not using them.

Vast.ai
`4090 ~$0.20–0.40/hr`

Marketplace of independent hosts. Cheapest tier is interruptible.

RunPod
`4090 ~$0.35–0.70/hr`

Per-second billing, so a fifteen minute run costs fifteen minutes.

Lambda
`A100 80GB ~$2.80/hr`

Fixed hourly rates, preconfigured PyTorch images, no egress fees.

Modal
`free tier + usage`

Serverless GPUs billed per second. Monthly free credit on the free plan.

Colab Pro
`$9.99–49.99/mo`

L4 and A100 through compute units. Easiest step up from the free tier.

Student credit programs
`free credits`

Azure for Students, AWS Educate, GitHub Student Developer Pack.

If the project only finishes because you paid for compute, the scope is too big. Come talk to me.

Rates are approximate as of September 2026 and change constantly

What to measure

Before and after every optimization, on the same configuration

PHASES TO TIME	METRICS TO REPORT
Data preprocessing	Time per iteration, throughput
Data loading	GPU utilization and achieved bandwidth
Host-to-device transfer	Peak memory footprint
Training step	Serving: TTFT, inter-token latency, goodput
Inference: prefill and decode	Task quality or correctness

TOOLS

PyTorch Profiler NVIDIA Nsight Systems Serving-framework logs, for example vLLM Hardware-specific profilers

Report the workload and configuration with every number.

Project ideas

Your own proposal is welcome if it has comparable depth.

Directions 1 to 3

01

torch.compile and graph optimization

Study where compilation helps and where it does not. Report on the Inductor backend and what it changes about the generated code.

Which parts of this model benefit from torch.compile, and where do graph breaks or recompilation erase the gain?

02

LLM inference, fine-tuning and serving

Take a small language model and either optimize its inference or fine-tune it for a task. Serving work can use vLLM, TGI or SGLang.

How do context length and batch size change TTFT, token latency, memory use and goodput for this model?

03

GPU infrastructure and AI accelerators

Run the same workload on more than one platform: cloud GPUs, the Columbia cluster, TPUs, or edge devices such as Jetson.

How does the same workload behave across two accelerator platforms, and which resource explains the difference?

Directions 4 to 6

04

Performance analysis study

`profile → select optimization → measure → explain`

Profile across preprocessing, loading, training and inference, then apply two or three techniques: dataloader changes, distributed training, mixed precision or quantization, distillation, sparsity, FlashAttention, speculative decoding, KV-cache optimizations.

05

Reproduce and extend a paper

Pick a paper from NeurIPS, ICLR, ICML, MLSys or CVPR on a topic from this course. Reproduce the result on hardware you have access to, then try to improve on it. Compare your numbers against the published ones and account for the differences.

06

Analog in-memory computing

Use the open AIHWKIT to study how a model behaves on analog devices: PCM, RRAM, ECRAM. Run hardware-aware training and measure the effect on drift. Results can be contributed upstream as an example.

github.com/IBM/aihwkit · documentation · example notebooks

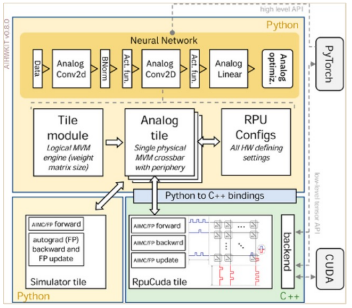


Figure: AIHWKIT v0.8.0 stack

More project ideas

Current topics that still fit on one GPU in six weeks

07

Speculative decoding

Pair a small draft model with a larger target model and vary the draft size.

How does acceptance rate trade against end-to-end latency?

08

KV-cache quantization

Run the cache in FP8 or INT4 at long context and watch memory and output quality together.

What does the memory saving cost in quality?

09

Dataloader bottleneck study

Profile preprocessing, workers, pinning and transfer before touching the model.

How much of the step time never reaches the GPU?

10

torch.compile mode comparison

Compare default, reduce-overhead and max-autotune, and audit where graph breaks appear.

Which mode wins for this model, and why?

11

Energy per token

Sample power during inference and report joules per token alongside latency.

Does the fastest configuration also use the least energy?

12

On-device inference

Run a quantized model on Jetson or Apple Silicon and measure sustained throughput.

How much does thermal throttling change the numbers?

Pick one and narrow it in office hours. Each of these has a baseline you can measure in week one.

Your own idea is welcome

Before committing to it, answer two questions.

01 What measurement will settle the question?

02 Can your team obtain that result within the project window?

Bring the idea to office hours early. We can usually make a good idea measurable by narrowing the workload or the question.

Deliverables

Code	Python or Jupyter notebooks in a GitHub repository. The README must include how to run and reproduce your results, dependencies, and environment setup.
Contribution	Repository history and the individual contribution statement are used to document each member's work.
Experiment tracking	**Required.** Configuration, metrics and hardware for every run, in a record the teaching staff can open. Tools are on the next slide.
Report	IEEE conference format, written in LaTeX. Overleaf and the IEEE template.
Presentation	Fifteen minutes plus questions, using the class template. Every member presents part of it.

Set up tracking in week one. You cannot reconstruct the before numbers at the end.

Track every experiment

`configuration + hardware + metric + artifact`

Use a tool that records runs and produces a shareable dashboard, or an equivalent record, for the teaching staff. Worth 20% of the project grade.

DEFAULT

Weights & Biases

Free academic tier, and the one the teaching staff knows best.

ALSO ACCEPTED

MLflow · ClearML · Aim · Comet · Neptune · DVCLive

Whatever tool you use, the final record must make run configurations and comparisons accessible to the teaching staff.

EVERY RUN RECORDS

`hardware · dtype · batch/context warm-up · repetitions · baseline`

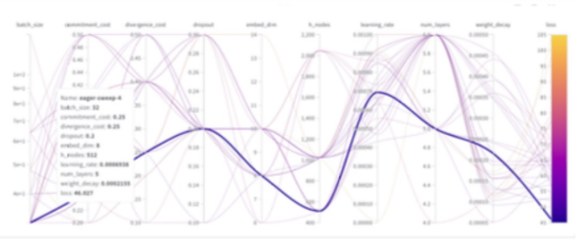


Figure 6: Hyperparameter Search.

`Sweep from a prior project report · Bhavanam, Safi`

Report structure

IEEE conference format, written in LaTeX on Overleaf

1 **Abstract**
Goals, method, key results

2 **Introduction**
Background and motivation

3 **Model and data**
What you used and why

4 **Training and profiling methodology**
Setup, tools, data handling

5 **Performance-tuning methodology**
Which techniques, and why those

6 **Experimental results**
Matched before-and-after configurations

7 **Conclusion**
Findings, limitations, future work

State limitations explicitly. Say where the result does not hold.

Project grading

5%	Project proposal
25%	Quality of code, profiling, documentation and reproducibility of the results
20%	Experiment tracking and dashboard
25%	Technical contribution, novelty, performance methodology and analysis
15%	Quality of the final report
10%	Presentation and questions

UP TO 10% BONUS, ON TOP OF THE ABOVE

A technical blog post or article with a published link. A merged pull request to an open-source project. Results that are publishable at a workshop o conference. A public repository, dataset or tool that others can use.

Example from HPML: kaoutarelmaghraoui.com/blog/hpml-industrial-mentorship-arxiv

Timeline

Office hours are available throughout for scoping and proposal discussions

WEEK 4 · 9 OCT

Proposal due

9 October. Team roles, model, data, hardware, the measurement you intend to make.

WEEK 9 · ENDS 13 NOV

Mid-point checkpoint

Baseline running, profile in hand, first optimization under way.

WEEK 13 · ENDS 11 DEC

Report and code due

Repository, tracking record and final report submitted.

FINALS · 17–23 DEC

Presentations

Fifteen minutes plus questions, during the finals period.

If the baseline is not running by the midpoint, reduce scope before adding more techniques.

Exemplar projects from last spring

EXAMPLE INDUSTRY-MENTORED PROJECTS

FULL STORY

HPML Industrial Mentorship: From Course Projects to Research

kaoutarelmaghraoui.com/blog/hpml-industrial-mentorship-arxiv

How those projects worked

Industry or research problem

↓

Student proposal

↓

Measured systems study

↓

Paper or open-source result

All four projects built on AssetOpsBench , IBM's open benchmark for AI agents in industrial asset operations. It is an active research benchmark and codebase, so any speedup was measured against a system other people are still developing.

A pitch had to show the team understood the problem, had a plausible approach, and knew how they would measure whether it worked.

The rest was ordinary course method: profile first, pick a technique you can defend against the profile, and log the before and after numbers so others can check them.

PRIOR PROJECT · SPRING 2026 · MERCHANT, VEERA, GOYLA, BHURE

Temporal caching for an agent workflow

QUESTION

How can we reuse prior results without returning stale answers?

Caching by text similarity is unsafe when the answer depends on time, on the asset, and on the sensor parameters. Their temporal cache classifies each query by how its answer depends on time, into volatile, static, relative and anchored, and reuses a stored result only when it is still valid. Volatile live-state queries skip the cache. Separately, a tool-discovery cache and dependency-aware parallel tool calls speed up the pipeline without changing its output.

1.67×

End-to-end, from the workflow optimizations alone, with identical results and about 40% lower median latency

30.6×

Median on the subset of queries that hit the temporal cache. Combined end-to-end median was about 3.48×

Limit: semantic similarity alone is not a reliable validity test. Hit-decision F1 reached about 0.64, and cross-parameter false positives remain the hard case.

PRIOR PROJECT · SPRING 2026 · CHETHAN KUMAR, KANATHUR

Generating benchmark scenarios at scale

The benchmark's 141 scenarios were authored by hand by subject-matter experts. That does not scale, and it limits how quickly new asset classes can be added.

THE GENERATOR

Asset profiling
Target asset and its available tools

Coverage allocation
Scenarios budgeted across cases

Generation
Candidates produced in batches

Validate and repair
Failures on schema, tool domain, grounding or duplication are repaired and re-checked

~8× Warm-cache throughput against the naive baseline at 50 scenarios

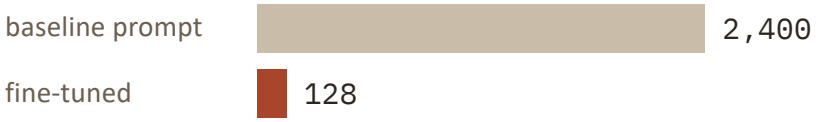
74.2 ± 1.9 Composite quality, against 73.8 ± 3.0 for the hand-tuned baseline

Limit: the intervals overlap, so the result supports preserved measured quality; it does not establish statistical equivalence. One model backend was tested, and expert review is still recommended.

PRIOR PROJECT · SPRING 2026 · SHEMLA, YAKOBE, AGARWAL

Moving a tool catalog from the prompt into the weights

TOOL-DESCRIPTION TOKENS PER CALL



Tool descriptions only. Each request still carries the query, so total input length fell about 82.6%.

For a fixed, known set of tools, the catalog can be stored in the model instead of resent on every call. Using 8-bit QLoRA they trained Gemma 4 E4B and Qwen3-4B on about 1,700 examples, then compared planning quality against a baseline that still receives the full descriptions.

0.65 vs 0.47 Agent-trajectory F1, best fine-tuned Gemma against the informed baseline. Qwen3-4B ran about 2.5× faster on 62% less memory

Limit: efficiency measured at model level on one A100, not end-to-end. Higher adapter rank improved planning but eroded pretraining accuracy, and the catalog is now fixed in the weights.

What these projects did well

Separate your numbers

A speedup on the subset that hits a cache is not the end-to-end speedup. Report both, and say which is which.

State the limit

Each of these papers says where the result does not hold: one hardware platform, one model backend, a metric that cannot resolve the difference.

Keep the scope small

Each is one question about one system. That is why they finished in a semester and held up afterwards.

READ THE FULL STORY

HPML Industrial Mentorship: From Class Projects to Research

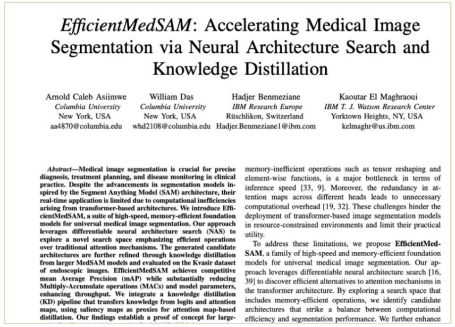
The article follows how these course projects developed into research results.

kaoutarelmaghraoui.com/blog/hpml-industrial-mentorship-arxiv

A technical article like this one is also one of the bonus routes on the grading slide.

Other projects from previous semesters

Students have worked on vision, LLM inference, analog AI, compilers and architecture



EfficientMedSAM: Accelerating Medical Image Segmentation via Neural Architecture Search and Knowledge Distillation

Arnold Caleb Asiimwe, *Columbia University*, New York, USA, aa4870@columbia.edu
William Das, *Columbia University*, New York, USA, wbd2108@columbia.edu
Hadjer Benmeziane, *IBM Research Europe*, Rüschlikon, Switzerland, Hadjer.Benmeziane1@ibm.com
Kaoutar El Maghraoui, *IBM T. J. Watson Research Center*, Yorktown Heights, NY, USA, kelmaghr@us.ibm.com

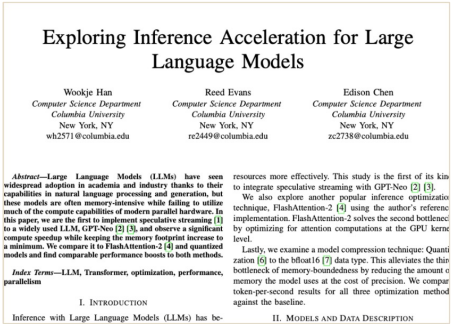
***Abstract*—Medical image segmentation is crucial for precise diagnosis, treatment planning, and disease monitoring in clinical practice. Despite the advancements in segmentation models inspired by the Segment Anything Model (SAM) architecture, their real-time application is limited due to computational inefficiencies arising from transformer-based architectures. We introduce EfficientMedSAM, a suite of high-speed, memory-efficient foundation models for universal medical image segmentation. Our approach leverages differentiable neural architecture search (NAS) to explore a novel search space emphasizing efficient operations over traditional attention mechanisms. The generated candidate architectures are further refined through knowledge distillation from larger MedSAM models and evaluated on the Kvasir dataset of endoscopic images. EfficientMedSAM achieves competitive mean Average Precision (mAP) while substantially reducing Multiply-Accumulate operations (MACs) and model parameters, enhancing throughput. We integrate a knowledge distillation (KD) pipeline that transfers knowledge from logits and attention maps, using saliency maps as proxies for attention map-based distillation. Our findings establish a proof of concept for large-**

memory-inefficient operations such as tensor reshaping and element-wise functions, is a major bottleneck in terms of inference speed [33, 9]. Moreover, the redundancy in attention maps across different heads leads to unnecessary computational overhead [19, 32]. These challenges hinder the deployment of transformer-based image segmentation models in resource-constrained environments and limit their practical utility.

To address these limitations, we propose **EfficientMedSAM**, a family of high-speed and memory-efficient foundation models for universal medical image segmentation. Our approach leverages differentiable neural architecture search [16, 39] to discover efficient alternatives to attention mechanisms in the transformer architecture. By exploring a search space that includes memory-efficient operations, we identify candidate architectures that strike a balance between computational efficiency and segmentation performance. We further enhance

EfficientMedSAM

Architecture search and knowledge distillation for medical image segmentation. Published at the IEEE Edge Conference 2024.



Exploring Inference Acceleration for Large Language Models

Wookje Han, *Computer Science Department, Columbia University*, New York, NY, wh2571@columbia.edu
Reed Evans, *Computer Science Department, Columbia University*, New York, NY, re2449@columbia.edu
Edison Chen, *Computer Science Department, Columbia University*, New York, NY, ze2738@columbia.edu

***Abstract*—Large Language Models (LLMs) have seen widespread adoption in academia and industry thanks to their capabilities in natural language processing and generation, but these models are often memory-intensive while failing to utilize much of the compute capabilities of modern parallel hardware. In this paper, we are the first to implement speculative streaming [1] to a widely used LLM, GPT-Neo [2] [3], and observe a significant compute speedup while keeping the memory footprint increase to a minimum. We compare it to FlashAttention-2 [4] and quantized models and find comparable performance boosts to both methods.**

***Index Terms*—LLM, Transformer, optimization, performance, parallelism**

I. Introduction

Inference with Large Language Models (LLMs) has be-

resources more effectively. This study is the first of its kin to integrate speculative streaming with GPT-Neo [2] [3].

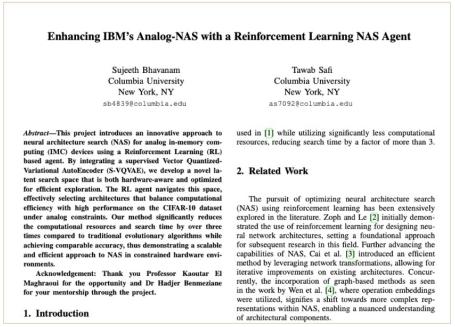
We also explore another popular inference optimizatio technique, FlashAttention-2 [4] using the author's referenc implementation. FlashAttention-2 solves the second bottlenec by optimizing for attention computations at the GPU kerne level.

Lastly, we examine a model compression technique: Quanti zation [6] to the bfloat16 [7] data type. This alleviates the thir bottleneck of memory-boundedness by reducing the amount o memory the model uses at the cost of precision. We compar token-per-second results for all three optimization method against the baseline.

II. Models and Data Description

Inference acceleration for LLMs

Speculative streaming on GPT-Neo, compared against FlashAttention-2 and a quantized model.



Enhancing IBM's Analog-NAS with a Reinforcement Learning NAS Agent

Sujeeth Bhavanam, Columbia University, New York, NY, sb4839@columbia.edu
Tawab Safi, Columbia University, New York, NY, as7092@columbia.edu

***Abstract*—This project introduces an innovative approach to neural architecture search (NAS) for analog in-memory computing (IMC) devices using a Reinforcement Learning (RL) based agent. By integrating a supervised Vector Quantized-Variational AutoEncoder (S-VQVAE), we develop a novel latent search space that is both hardware-aware and optimized for efficient exploration. The RL agent navigates this space, effectively selecting architectures that balance computational efficiency with high performance on the CIFAR-10 dataset under analog constraints. Our method significantly reduces the computational resources and search time by over three times compared to traditional evolutionary algorithms while achieving comparable accuracy, thus demonstrating a scalable and efficient approach to NAS in constrained hardware environments.**

Acknowledgement: Thank you Professor Kaoutar El Maghraoui for the opportunity and Dr Hadjer Benmeziane for your mentorship through the project.

1. Introduction

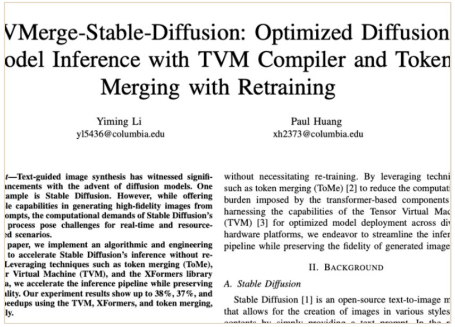
used in [1] while utilizing significantly less computational resources, reducing search time by a factor of more than 3.

2. Related Work

The pursuit of optimizing neural architecture search (NAS) using reinforcement learning has been extensively explored in the literature. Zoph and Le [2] initially demonstrated the use of reinforcement learning for designing neural network architectures, setting a foundational approach for subsequent research in this field. Further advancing the capabilities of NAS, Cai et al. [3] introduced an efficient method by leveraging network transformations, allowing for iterative improvements on existing architectures. Concurrently, the incorporation of graph-based methods as seen in the work by Wen et al. [4], where operation embeddings were utilized, signifies a shift towards more complex representations within NAS, enabling a nuanced understanding of architectural components.

Analog-NAS with an RL agent

Architecture search for analog in-memory devices, cutting search time by more than three times on CIFAR-10.



VMerge-Stable-Diffusion: Optimized Diffusion odel Inference with TVM Compiler and Token Merging with Retraining

Yiming Li, y15436@columbia.edu
Paul Huang, xh2373@columbia.edu

t—Text-guided image synthesis has witnessed signifi-ncements with the advent of diffusion models. One ample is Stable Diffusion. However, while offering le capabilities in generating high-fidelity images from ompts, the computational demands of Stable Diffusion's process pose challenges for real-time and resource-d scenarios.
paper, we implement an algorithmic and engineering to accelerate Stable Diffusion's inference without re-Leveraging techniques such as token merging (ToMe), r Virtual Machine (TVM), and the XFormers library a, we accelerate the inference pipeline while preserving ility. Our experiment results show up to 38%, 37%, and eedups using the TVM, XFormers, and token merging, ly.

without necessitating re-training. By leveraging techni such as token merging (ToMe) [2] to reduce the computat burden imposed by the transformer-based components harnessing the capabilities of the Tensor Virtual Mac (TVM) [3] for optimized model deployment across di hardware platforms, we endeavor to streamline the infer pipeline while preserving the fidelity of generated image

II. Background

A. Stable Diffusion

Stable Diffusion [1] is an open-source text-to-image m that allows for the creation of images in various styles

TVMerge-Stable-Diffusion

TVM compilation, XFormers and token merging for diffusion inference, reported at up to 38% faster.

Resources

PROFILING AND TUNING

PyTorch Profiler recipe NVIDIA Nsight Systems PyTorch tuning guide TorchBench

MODELS, DATA, BENCHMARKS

Hugging Face models and datasets PyTorch vision datasets MLCommons and MLPerf AssetOpsBench

FINE-TUNING

PEFT library LoRA paper QLoRA paper

TRACKING AND WRITING

Weights & Biases MLflow Overleaf IEEE conference template

SERVING

vLLM Text Generation Inference SGLang DeepSpeed-Inference

ANALOG AI

AIHWKIT on GitHub Documentation Example notebooks Tutorial paper, APL Machine Learning

PROJECT SHOWCASE

HPML Industrial Mentorship

CourseWorks has the current list, and sample projects from prior semesters are there under Files. Bring scope questions to Ed or office hours, early rather than the week before the deadline.